EMMANUEL AJALA

AI/ML Systems Engineer — Distributed Training | LLM Inference | Retrieval-Augmented Generation

Lagos, Nigeria | ajalae2@gmail.com | github.com/manofvalour | linkedin.com/in/emmanuelaj

SUMMARY

AI/ML systems engineer focused on distributed training, LLM inference, and retrieval-augmented generation. Builds systems from first principles — custom multi-agent RAG pipelines, transformer architectures implemented from scratch, and applied study of GPU performance (roofline analysis, parallelism strategies, inference serving). Comfortable owning a project end-to-end: design, implementation, debugging, and evaluation.

TECHNICAL SKILLS

Languages & Frameworks: Python, PyTorch, FastAPI

Distributed Training: Tensor parallelism, DDP, ZeRO/FSDP, multi-GPU training, communication/compute-bound analysis

LLM Inference: KV caching, prefill/decode optimization, grouped-query attention, disaggregated inference serving, roofline analysis

Retrieval-Augmented Generation: Query expansion (HyDE, multi-query), reranking, retrieval (Qdrant), multi-agent pipelines, hallucination detection

Infrastructure & Tools: Docker, Redis, Prometheus, Grafana, LangSmith, Weights & Biases, Linux (Arch/Hyprland), Git

PROJECTS

Ask Your Doc — Multi-Agent RAG System | github.com/manofvalour/Multiagent_RAG_system

- Designed and built a 7-stage retrieval-augmented pipeline largely from scratch (query expansion, retrieval, reranking, consensus, claim verification, confidence scoring)
- Built the query expansion stage with HyDE and multi-query running in parallel, and integrated Qdrant for vector retrieval
- Built async pipeline stages, including a multi-generator step that samples candidate answers at different temperatures, followed by a consensus stage that selects the best output
- Built a claim-verification stage that checks generated claims against retrieved source documents, and a confidence-scoring stage that classifies output into low/medium/high hallucination risk
- Running a 16-configuration experiment (HyDE, multi-query, or both; 1 or 3 generators; reranker on or off) to find the setup that best balances latency, cost, and output quality
- Deployed the full system with a FastAPI backend, Redis caching, and a Prometheus/Grafana/LangSmith observability stack

Model Architecture Replication — LSTM, GPT-2, MoE, Encoder-Decoder Transformer | github.com/manofvalour/Models_Replication

- Built an encoder-decoder Transformer from scratch for English-to-French translation and trained it with PyTorch DDP across multiple GPUs
- Debugged real distributed-training failures: multiprocessing pickling errors, DataLoader out-of-memory issues, experiment-tracking step conflicts, and mixed-precision (fp16/bf16) compatibility issues
- Implemented GPT-2 from scratch, referencing nanoGPT and Hugging Face's implementation, and verified correctness by loading Hugging Face's pretrained weights and confirming matching outputs
- Built a Mixture-of-Experts module from scratch (router plus multiple expert layers)
- Built an LSTM model from scratch

Distributed Training & Inference — Applied Study

- Derived the roofline/compute-cost formula for transformer training from first principles while working through the JAX/Google DeepMind scaling book
- Studied how tensor parallelism, DDP, and ZeRO/FSDP partition weights, gradients, and optimizer state, and identified when each becomes compute-bound versus communication-bound
- Studied inference-time performance: why prefill is compute-bound while decode is bandwidth-bound due to sequential token generation, and inference optimization techniques including KV caching, grouped-query attention, and disaggregated prefill/decode serving

EDUCATION

B.Eng., Civil Engineering